Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО

Отчёт

По лабораторной работе №2

Вариант: 3124123

Студенты:

Долинный М. В.

Антипин Г. В.

группа *P3332*

Преподаватель:

Чупанов А. А.

г. Санкт-Петербург, 2026 г.

Описание задания

Лабораторная работа #2

Провести интеграционное тестирование программы, осуществляющей вычисление системы функций (в соответствии с вариантом).

Введите вариант: 3124123

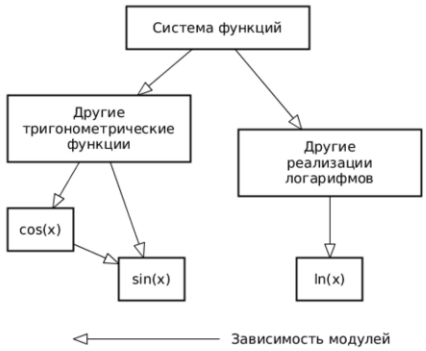
$$\begin{cases} \left(\left(\left(\left(\frac{\frac{\cot(x)}{\csc(x)}}{\sin(x)}\right) - \csc(x)\right)^3\right)^2\right) & \text{if } x \le 0 \\ \left(\frac{\left(((\log_{10}(x)+\ln(x))+\log_{10}(x)) - \left(\log_3(x)^3\right)\right)\cdot\log_{10}(x)}{\frac{\log_3(x)+((\log_5(x)-\log_2(x))\cdot\log_2(x))}{(\log_{10}(x)-\log_{10}(x))-\ln(x)}}\right) & \text{if } x > 0 \end{cases}$$

x <= 0 : (((((cot(x) / csc(x)) / sin(x)) - csc(x)) ^ 3) ^ 2)

x > 0 : (((((log_10(x) + ln(x)) + log_10(x)) - (log_3(x) ^ 3)) * log_10(x)) / ((log_3(x) + ((log_5(x) - log_2(x)) * log_2(x))) / ((log_10(x) - log_10(x)) - ln(x))))

Правила выполнения работы:

1. Все составляющие систему функции (как тригонометрические, так и логарифмические) должны быть выражены через базовые (тригонометрическая зависит от варианта; логарифмическая - натуральный логарифм).
2. Структура приложения, тестируемого в рамках лабораторной работы, должна выглядеть следующим образом (пример приведён для базовой тригонометрической функции sin(x)):

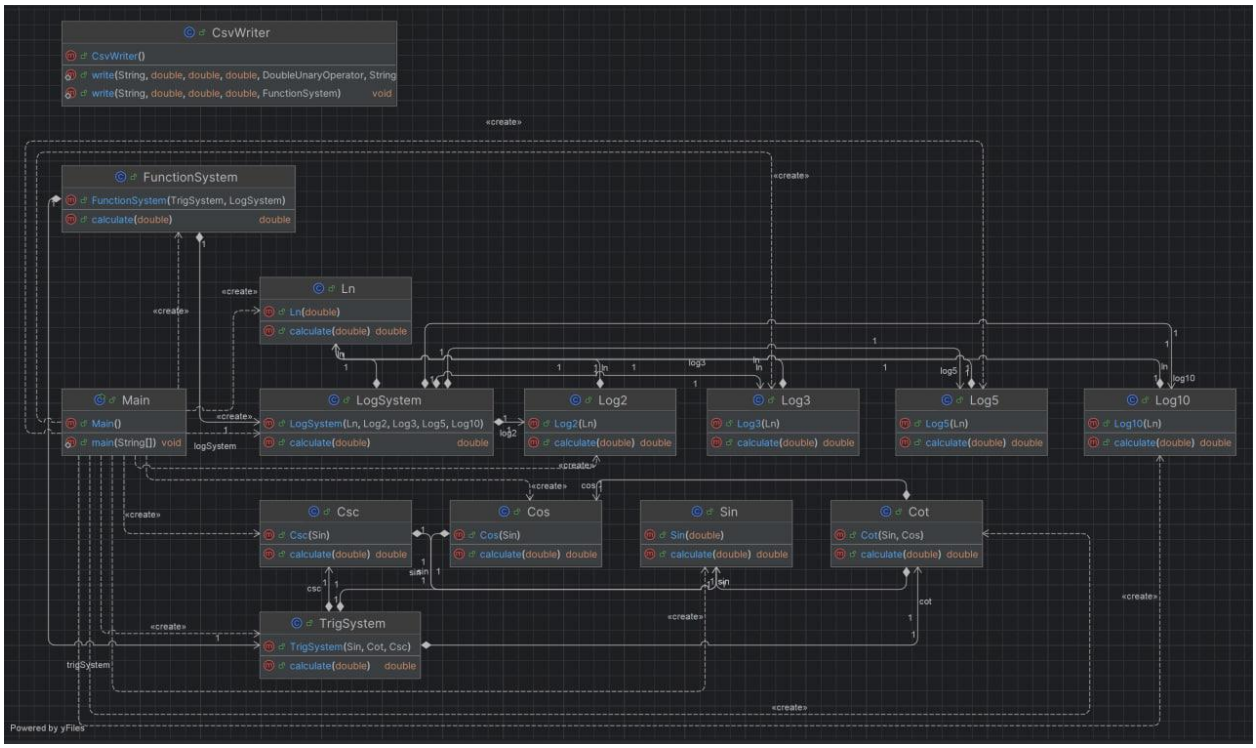


3. Обе "базовые" функции (в примере выше - sin(x) и ln(x)) должны быть реализованы при помощи разложения в ряд с задаваемой погрешностью. Использовать тригонометрические / логарифмические преобразования для упрощения функций ЗАПРЕЩЕНО.
4. Для КАЖДОГО модуля должны быть реализованы табличные заглушки. При этом, необходимо найти область допустимых значений функций, и, при необходимости, определить взаимозависимые точки в модулях.
5. Разработанное приложение должно позволять выводить значения, выдаваемое любым модулем системы, в csv файл вида «X, Результаты модуля (X)», позволяющее произвольно менять шаг наращивания X. Разделитель в файле csv можно использовать произвольный.

Порядок выполнения работы:

1. Разработать приложение, руководствуясь приведёнными выше правилами.
2. С помощью JUNIT5 разработать тестовое покрытие системы функций, проведя анализ эквивалентности и учитывая особенности системы функций. Для анализа особенностей системы функций и составляющих ее частей можно использовать сайт https://www.wolframalpha.com/.
3. Собрать приложение, состоящее из заглушек. Провести интеграцию приложения по 1 модулю, с обоснованием стратегии интеграции, проведением интеграционных тестов и контролем тестового покрытия системы функций.

UML


CsvWriter
CsvWriter()
write(String, double, double, double, DoubleUnaryOperator, String
write(String, double, double, double, FunctionSystem) void
FunctionSystem
FunctionSystem(TrigSystem, LogSystem)
calculate(double) double
Ln
Ln(double)
calculate(double) double
Main
Main()
main(String[]) void
LogSystem
LogSystem(Ln, Log2, Log3, Log5, Log10)
calculate(double) double
Log2
Log2(Ln)
calculate(double) double
Log3
Log3(Ln)
calculate(double) double
Log5
Log5(Ln)
calculate(double) double
Log10
Log10(Ln)
calculate(double) double
Csc
Csc(Sin)
calculate(double) double
Cos
Cos(Sin)
calculate(double) double
Sin
Sin(double)
calculate(double) double
Cot
Cot(Sin, Cos)
calculate(double) double
TrigSystem
TrigSystem(Sin, Cot, Csc)
calculate(double) double
«create»
logSystem
trigSystem
ln
log2
log3
log5
log10
cos
sin
csc
cot
Powered by yFiles

Описание тестового покрытия с обоснованием его выбора

Тригонометрические функции

В тригонометрическом блоке используются функции sin(x), cos(x), cot(x) и csc(x). Выбор тестов основан на аналитических значениях функций, их периодичности и наличии точек разрыва.

- **Характерные точки:**
  Проверяются значения 0, π/2, -π/2, π, π/3, π/4.
  Эти точки выбраны, потому что для них известны точные значения функций (например, sin(π/2) = 1, cos(π) = -1, cot(π/4) = 1). Это позволяет использовать их как опорные для проверки корректности реализации.
- **Произвольные значения:**
  Используется значение x = -1.
  Оно необходимо для проверки корректности вычислений вне “табличных” точек и подтверждает, что функции работают на обычных аргументах.
- **Отрицательные значения:**
  Проверяются отдельно (например, -1, -π/2), так как вся тригонометрическая ветвь системы используется при x ≤ 0. Это позволяет протестировать именно рабочую область системы.
- **Периодичность и нормализация аргумента (sin):**
  Проверяются значения 2π, 3π/2, -3π/2.
  Эти тесты подтверждают, что реализация корректно нормализует аргумент к диапазону [-π, π] перед вычислением через ряд Тейлора.
- **Особые точки (разрывы):**
  Для функций cot(x) и csc(x) проверяются точки 0 и π, где sin(x) = 0.
  В этих точках функции не определены, поэтому ожидается выброс исключения. Это проверяет корректную обработку деления на ноль.

Таким образом, покрытие включает опорные значения, произвольные точки, отрицательную область, периодичность и все ключевые точки разрыва тригонометрических функций.

Логарифмические функции

Логарифмический блок включает функции ln(x), log2(x), log3(x), log5(x) и log10(x). Все они реализованы через натуральный логарифм, поэтому тестирование строится вокруг свойств ln(x).

- **Характерные точки:**
  - x = 1 => ln(1) = 0 (граничная точка)
  - x = e => ln(e) = 1
  - x = base => log_base(base) = 1 (например, log2(2), log3(3))

Эти значения позволяют проверить базовые тождества логарифмов.

- **Обычные значения из области x > 1:**
  Используются 2, 7.
  Они позволяют проверить корректность вычислений на произвольных аргументах.
- **Дробные значения (0 < x < 1):**
  Используется x = 0.5.
  В этом диапазоне логарифмы отрицательны, поэтому такие тесты проверяют корректность знака и вычислений.
- **Некорректные аргументы:**
  Проверяются x = 0 и отрицательные значения.
  Логарифмы не определены при x ≤ 0, поэтому ожидается выброс исключения.

Таким образом, тесты покрывают границы области определения, характерные значения, разные интервалы ((0,1) и (1,+∞)) и обработку ошибок.

Система функций

Система функций состоит из двух ветвей:

- тригонометрическая - при x ≤ 0
- логарифмическая - при x > 0

Тестирование построено с учётом этого разбиения.

- **Проверка выбора ветви:**
  - x = -1 => используется тригонометрическая часть
  - x = 2 => используется логарифмическая часть

  Это проверяет корректность маршрутизации аргумента.

- **Граничная точка x = 0:**
  При x = 0 система переходит в тригонометрическую ветвь, где возникает деление на ноль (sin(0) = 0).
  Ожидается исключение - проверка корректной обработки границы.
- **Особая точка x = 1:**
  При x = 1 используется логарифмическая ветвь, но ln(1) = 0, что приводит к делению на ноль в формуле.
  Ожидается исключение.
- **Типичные значения:**
  - x = 2 (область x > 1)
  - x = 0.5 (область 0 < x < 1)

Эти тесты проверяют корректность вычислений в разных частях области определения логарифмической ветви.

- **Тригонометрическая часть отдельно:**
  - x = -1 - произвольное значение
  - x = -π/2 - опорная точка, где результат легко вычисляется аналитически
  - x = -π, 0 - точки разрыва

Таким образом, покрытие проверяет:

- корректность вычислений в обеих ветвях,
- правильность переключения между ними,
- обработку граничных и особых точек.

Тестовое покрытие

Как мы можем видеть, все работает корректно:

Test Summary

44	0	0	0.148s
tests	failures	ignored	duration

100% successful

Packages | Classes

Package	Tests	Failures	Ignored	Duration	Success rate
ru.traphouse.functions.log	12	0	0	0.075s	100%
ru.traphouse.functions.trig	21	0	0	0.023s	100%
ru.traphouse.system	11	0	0	0.050s	100%

Generated by Gradle 8.14 at 20 мар. 2026 г., 00:02:38

lab2

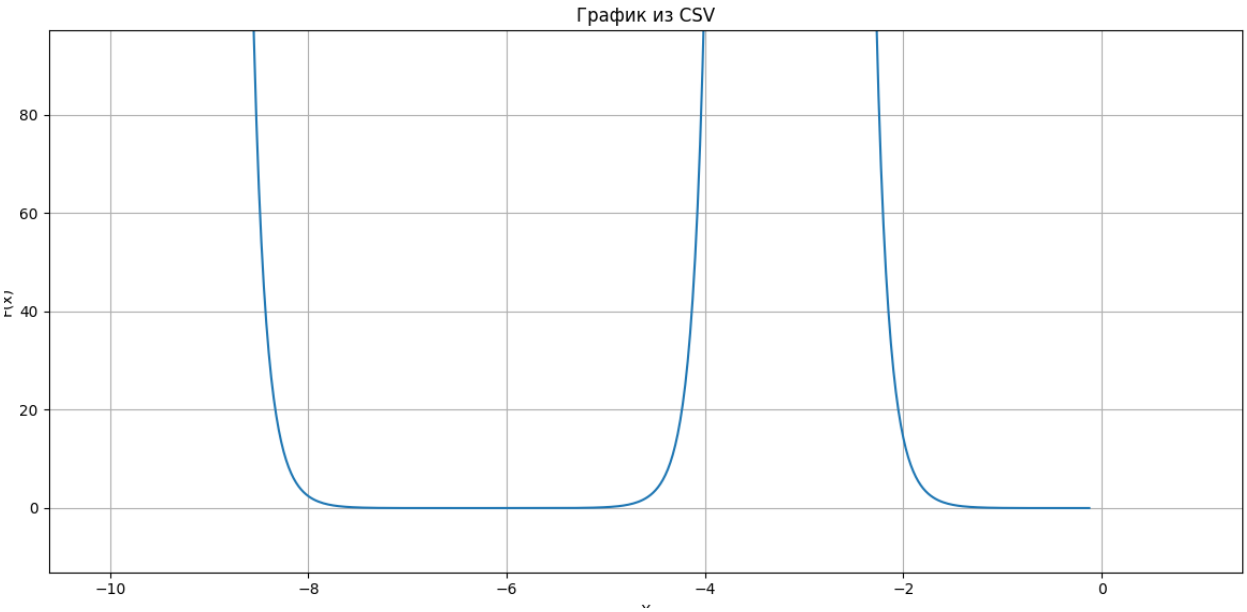
lab2

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ru.traphouse		0 %		n/a	4	4	22	22	4	4	1	1
ru.traphouse.util		0 %		0 %	4	4	12	12	3	3	1	1
ru.traphouse.system		100 %		100 %	0	8	0	36	0	6	0	3
ru.traphouse.functions.log		100 %		100 %	0	12	0	35	0	10	0	5
ru.traphouse.functions.trig		100 %		100 %	0	13	0	33	0	8	0	4
Total	179 of 613	70 %	2 of 20	90 %	8	41	34	138	7	31	2	14

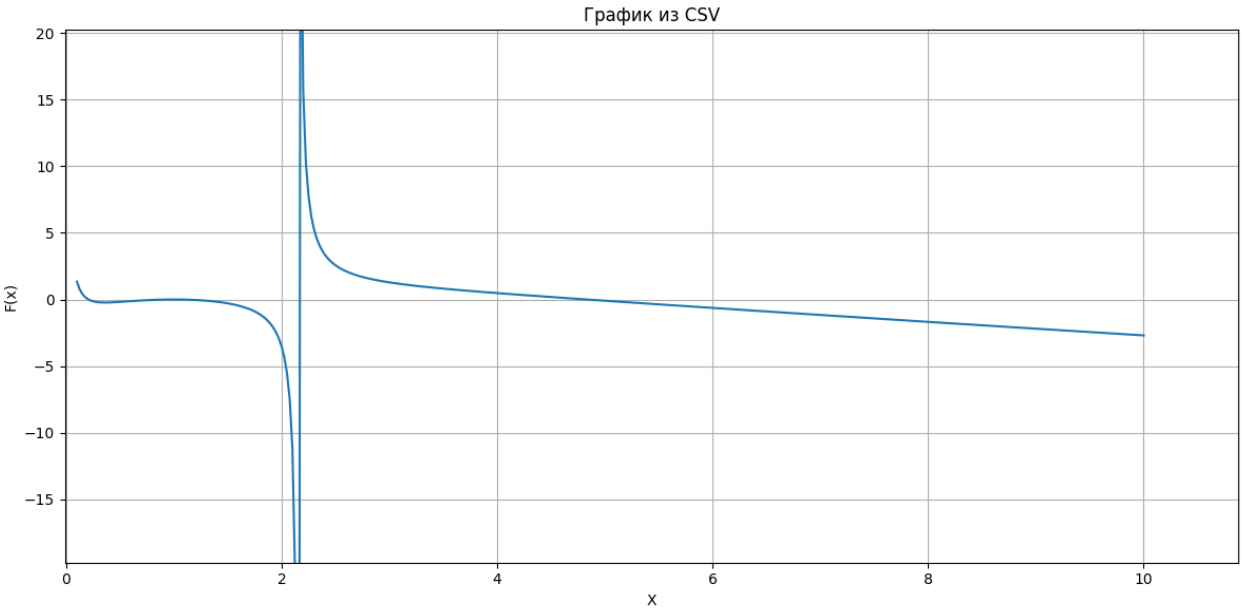
Итоговое тестовое покрытие.

Графики

trig



Log



System

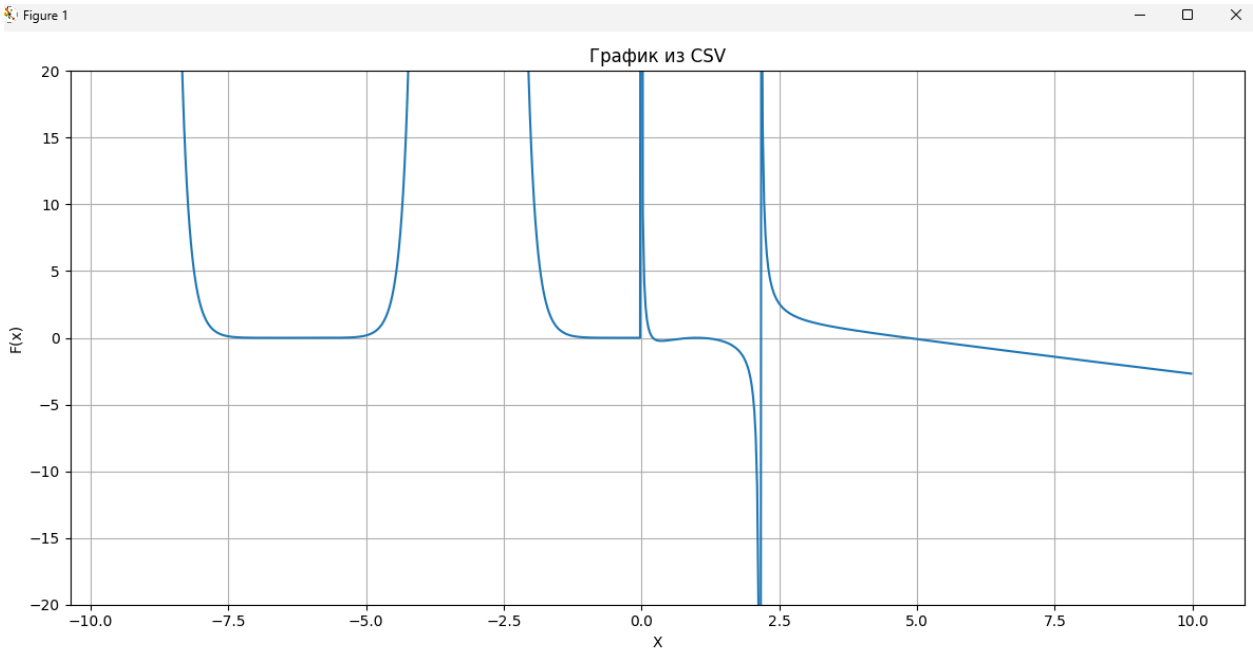


Figure 1
График из CSV
20
15
10
5
0
−5
−10
−15
−20
F(x)
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
X

Выводы

В ходе выполнения лабораторной работы была реализована система функций,основанная на разложении в ряд. Проведено модульное и интеграционное тестирование с использованием JUnit 5 и Mockito, включая анализ граничных значений классов эквивалентности. Кроме того я научился тестировать приложения, работающие с дисплеем, получил базовые навыки CI. Выгрузка данных в CSV построенные графики подтвердили корректность вычислений. Работа позволила закрепить навыки интеграционного тестирования, работы с численными методами и проектирования модульных приложений.